

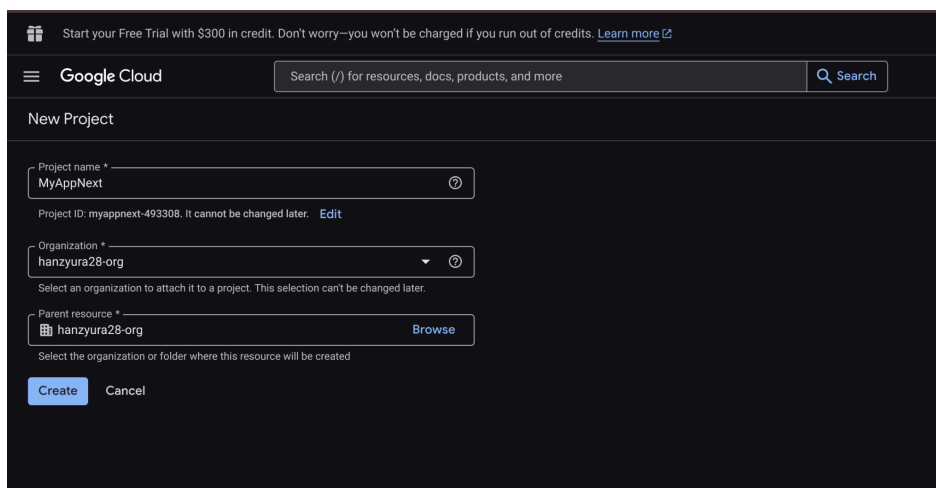


Mata Kuliah : Pemrograman Framework
Program Studi : D4 – Teknik Informatika
Semester : 6

Kelas : TI-3C
NIM : 2341720165
Nama : Fandy Wahyu Hanzura
Jobsheet Ke- : 160

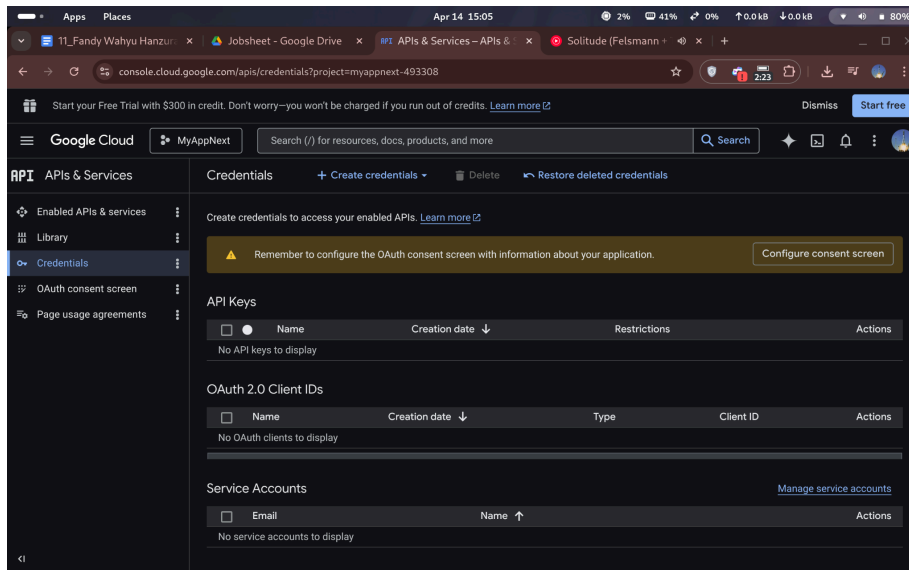
Laporan Jobsheet

Praktikum 1



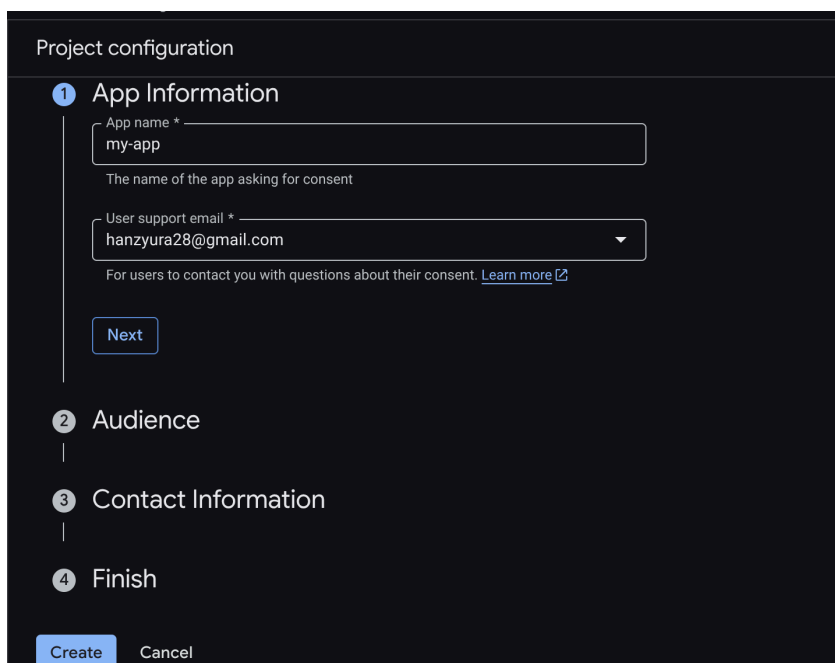
1.

- Membuat proyek baru pada google cloud dengan menggunakan nama My App Next



2.

- Masuk ke dalam halaman credentials dan memastikan sesuai dengan workspace yang ada yakni My Next App



3.



Project configuration

2 Audience

☐ Internal ?

Only available to users within your organization. You will not need to submit your app for verification. [Learn more about user type](#)

☒ External ?

Available to any test user with a Google Account. Your app will start in testing mode and will only be available to users you add to the list of test users. Once your app is ready to push to production, you may need to verify your app. [Learn more about user type](#)

Next

3 Contact Information

Project configuration

✓ App Information

✓ Audience

3 Contact Information

Email addresses *

hanyura28@gmail.com

These email addresses are for Google to notify you about any changes to your project.

Next

4 Finish

Create Cancel



- Melakukan pembuatan credentials pada halaman oauth

4.

- Menambahkan client web pada auth yang diarahkan ke halaman web



```
.env.local
1  FIREBASE_API_KEY
2  FIREBASE_AUTH_DOMAIN
3  FIREBASE_PROJECT_ID
4  FIREBASE_STORAGE_BUCKET
5  FIREBASE_MESSAGING_SENDER_ID
6  FIREBASE_APP_ID=
7  FIREBASE_MEASUREMENT_ID
8
9  REVALIDATE_TOKEN_EXPIRATION_SECONDS
10 NEXTAUTH_SECRET=
11
12 NEXT_AUTH_ID=766
13 NEXT_AUTH_SECRET
```

5.

```
.env.local  TS [...nextauth].ts M X
src > pages > api > auth > TS [...nextauth].ts > default
4  import { signIn } from "@utils/db/servicefirebase";
5  import GoogleProvider from "next-auth/providers/google";
6
7  export const authOptions: NextAuthOptions = {
8    session: {
9      strategy: "jwt"
10   },
11   secret: process.env.NEXTAUTH_SECRET,
12   providers: [
13     CredentialsProvider({
14       name: "Credentials",
15       credentials: {
16         email: { label: "Email", type: "email" },
17         password: { label: "Password", type: "password" }
18       },
19       async authorize(credentials) {
20         if (!credentials?.email || !credentials?.password) return null;
21         const user: any = await signIn(credentials.email);
22         if (user) {
23           const isValid = await bcrypt.compare(
24             credentials.password,
25             user.password,
26           );
27         }
28       }
29     })
30   ]
31 }
```



```
index.tsx .../navbar M  index.tsx .../login M X  # navbar.module.css M
src > views > auth > login > index.tsx > TampilanLogin
1  import Link from "next/link";
2  import style from "@views/auth/login/login.module.scss";
3  import { useState } from "react";
4  import { useRouter } from "next/router";
5  import { signIn, getSession } from "next-auth/react";
6
7  export default function TampilanLogin() {
8    const [isLoading, setIsLoading] = useState(false);
9    const [error, setError] = useState("");
10   const { push, query } = useRouter();
11
12   // Mengambil url tujuan setelah login berhasil (default ke "/")
13   const callbackUrl: any = query.callbackUrl || "/";
14
15   const handleSubmit = async (event: any) => {
16     event.preventDefault();
17     setError("");
18     setIsLoading(true);
19
20     try {
21       const res = await signIn("credentials", {
22         redirect: false,
23         email: event.target.email.value,
24         password: event.target.password.value,
25         callbackUrl,
26       });
27
28       if (!res?.error) {
29         setIsLoading(false);
30       }
31     }
32   }
33 }
```

Halaman Login

Email

Password

Login

sign in with google

Tidak punya akun? Ke Halaman Register



```
src > components > layouts > navbar > index.tsx > Navbar
4 : default function Navbar() {
10   <div className={styles.navbar__brand}>
11     MyApp
12   </div>
13
14   <div className={styles.navbar__right}>
15     {session ? (
16       <>
17         <div className={styles.navbar__user}>
18           Welcome, {session.user?.fullname || session.user?.name}
19           {session.user?.image && (
20             <img
21               src={session.user.image}
22               alt={session.user.fullname || "Profile"}
23               className={styles.navbar__user__image}
24               referrerPolicy='no-referrer'
25             />
26           )}
27         </div>
28         <button
29           className={` ${styles.navbar__button} ${styles["navbar__button__${session ? 'logged' : 'login'}"]} `}
30           onClick={() => signOut({ callbackUrl: "/" })}
31         >
32           {session ? "Sign Out" : "Sign In"}
33         </button>
34       </>
35     ) : (
36       <button
37         className={` ${styles.navbar__button} ${styles["navbar__button__${session ? 'logged' : 'login'}"]} `}
38       >
```



```
GET /api/auth/session 200 in 3ms (next.js: /15us, application-code: 2ms)
Session Callback Updated: {
  user: {
    name: 'Fandy Wahyu Hanzura',
    email: 'hanzyura28@gmail.com',
    image: 'https://lh3.googleusercontent.com/a/ACg8ocI0Xm34tmsMmpw5aN6ggnCSgsAsPSI9NjE5HYc_Dw0X_cBJf4se=s96-c',
    fullname: 'Fandy Wahyu Hanzura',
    role: 'member'
  },
  expires: '2026-05-14T13:14:18.173Z'
}
```

- Menambahkan halaman login menggunakan credential google pada halaman login



6.

```
src > utils > db > TS servicefirebase.ts > signInWithGoogle
1 import {
2   getFirestore,
3   collection,
4   getDocs,
5   getDoc,
6   doc,
7   query,
8   addDoc,
9   where,
10  updateDoc
11 } from "firebase/firestore";
12 import app from "@utils/db/firebase";
13 import bcrypt from "bcrypt";
14
15 const db = getFirestore(app);
16
17
18
19 > export async function retrieveProducts(collectionName: string) { ...
26 }
27
28 > export async function retrieveProductByID(collectionName: string, id: string) { ...
33 }
34
35
36 > export async function signUp(...
59 }
60
61 > export async function signIn(email: string) { ...
71 }
72
73 export async function signInWithGoogle(userData: any, callback: any) {
74   try {
75     const q = query(
76       collection(db, "users")
```

```
src > pages > api > auth > TS [...nextauth].ts > authOptions > providers > authorize
8 export const authOptions: NextAuthOptions = {
13   providers: [
14     CredentialsProvider({
15       name: "Credentials",
16       credentials: {
17         email: { label: "Email", type: "email" },
18         password: { label: "Password", type: "password" }
19       },
20
21       async authorize(credentials) {
22         if (!credentials?.email || !credentials?.password) return null;
23
24         const user: any = await signIn(credentials.email);
25
26         if (user) {
27           const isPasswordValid = await bcrypt.compare(
28             credentials.password,
29             user.password,
30           );
31           if (isPasswordValid) {
32             return {
33               id: user.id,
34               email: user.email,
35               fullname: user.fullname,
36               role: user.role,
37               image: user.image,
38             };
39           }
40         }
41         return null;
42       },
43     }),
44     GoogleProvider({
45       clientId: process.env.GOOGLE_CLIENT_ID || "",
46       clientSecret: process.env.GOOGLE_CLIENT_SECRET || ""
```




```
email: "hanzyura28@gmail.com"
fullname: "Fandy Wahyu Hanzura"
image: "https://lh3.googleusercontent.com/a/ACg8ocI0X
c"
password: "$2b$10$gRwG36mD3OkzGzAtaWi8N.b35gE
role: "admin"
type: "google"
```

- Menambahkan push sign up menggunakan google ke firebase sehingga data menjadi terupdate

Analisis Dan Diskusi

1. Apa perbedaan login credential dan login Google?

- Autentikasi menggunakan *credentials* adalah metode konvensional di mana sistem menyimpan dan memvalidasi data pengguna (email dan kata sandi) secara internal di dalam basis data aplikasi. Sebaliknya, Login Google menggunakan protokol **OAuth 2.0**, di mana proses verifikasi identitas didelegasikan kepada pihak ketiga (*Identity Provider*). Hal ini meningkatkan aspek keamanan karena aplikasi tidak perlu menyimpan kata sandi pengguna secara langsung

2. Mengapa data Google tetap perlu disimpan ke database?

- Penyimpanan data profil ke basis data lokal diperlukan untuk manajemen **otorisasi**. Meskipun Google menyediakan identitas pengguna, sistem aplikasi memerlukan basis data internal untuk menentukan tingkat hak akses (*Role-Based Access Control*), menyimpan



preferensi spesifik aplikasi, serta memungkinkan integrasi relasional dengan entitas data lain di dalam sistem (seperti riwayat transaksi atau aktivitas pengguna).

3. Apa fungsi JWT callback?

- *Callback* JWT berfungsi sebagai *middleware* untuk memproses dan memperkaya informasi di dalam token sebelum sesi autentikasi didefinisikan secara permanen. Di dalam alur ini, data dari penyedia layanan (seperti Google) dapat disinkronkan dengan basis data internal untuk menyertakan klaim tambahan, seperti peran pengguna (*role*), sehingga informasi tersebut tersedia secara konsisten di seluruh aplikasi.

4. Mengapa perlu multi-role?

- Sistem *multi-role* diterapkan untuk menjamin **keamanan dan integritas data** melalui pembatasan akses. Dengan membedakan hak akses antara *Admin* (pengelola sistem) dan *Member* (pengguna umum), aplikasi dapat memastikan bahwa fitur-fitur kritikal seperti manajemen pengguna atau konfigurasi sistem hanya dapat diakses oleh pihak yang berwenang, guna mencegah penyalahgunaan fungsi aplikasi.

5. Apa risiko jika tidak menyimpan user ke database?

- Tidak menyimpan data pengguna ke basis data internal akan mengakibatkan hilangnya kontrol administratif terhadap akun tersebut. Risiko teknis meliputi: ketidakmampuan untuk melakukan modifikasi hak akses secara manual, kesulitan dalam pelacakan aktivitas (*audit trail*), serta ketergantungan penuh pada penyedia layanan pihak ketiga yang dapat menghambat skalabilitas dan personalisasi fitur aplikasi di masa depan.

Analisis Dan Diskusi

- Tugas 1:



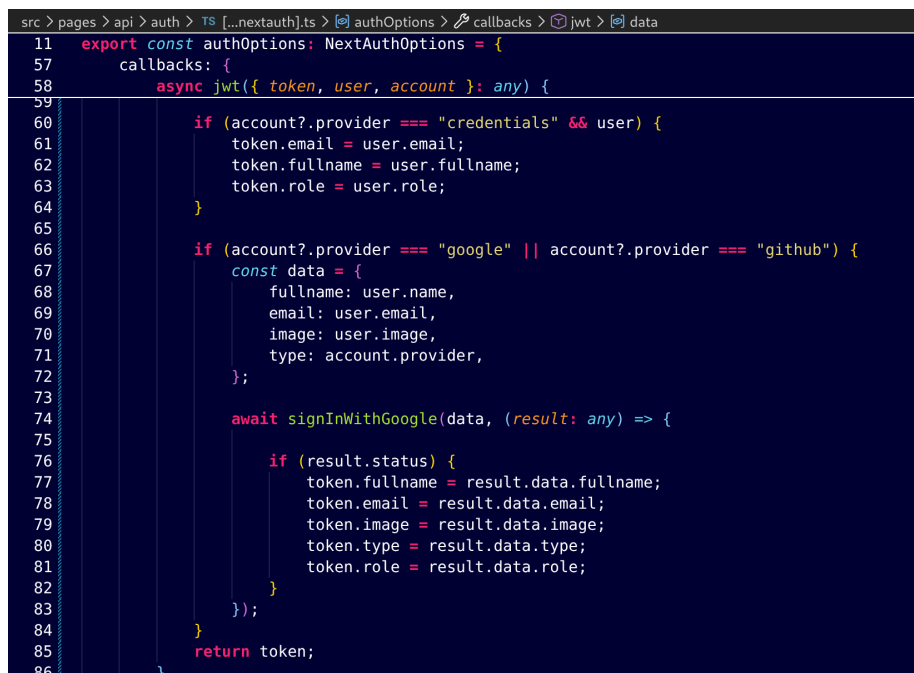
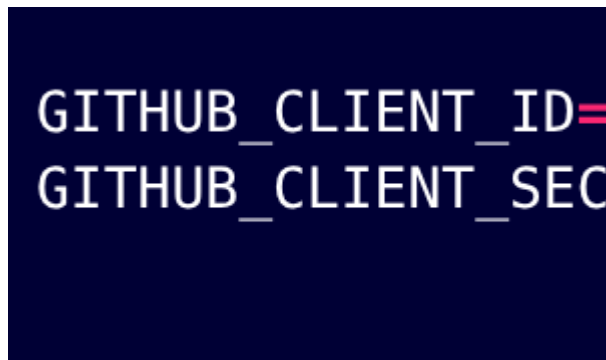
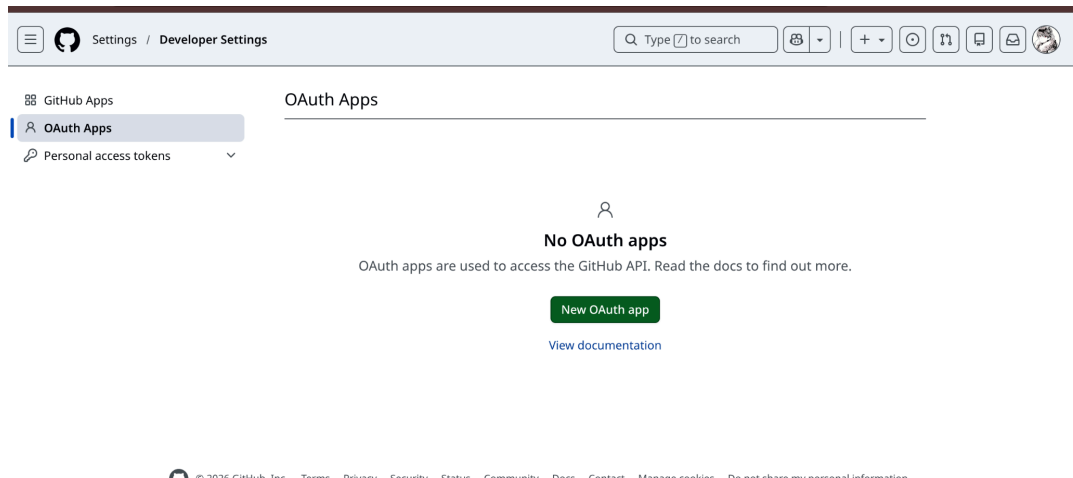
```
src > pages > editor > index.tsx > HalamanEditor
1  function HalamanEditor() {
2
3
4  .v className="admin">
5  <h1>Halaman Admin</h1>
6  <p>
7    Selamat datang di halaman Editor! Anda memiliki akses tidak penuh ke semua halaman
8  </p>
9  </div>
10
11
12
13
14
```

```
11 // 1. Jika rute butuh login tapi gak ada token, tendang ke login
12 if (requireAuth.includes(pathname) && !token) {
13   const loginUrl = new URL("/auth/login", req.url);
14   loginUrl.searchParams.set("callbackUrl", encodeURIComponent(req.url));
15   return NextResponse.redirect(loginUrl);
16 }
17
18 // 2. PROTEKSI KHUSUS: Hanya Admin yang boleh masuk rute /admin
19 if (pathname.startsWith("/admin") && token?.role !== "admin") {
20   return NextResponse.redirect(new URL("/", req.url));
21 }
22
23 if (pathname.startsWith("/editor") && token?.role !== "editor" && token?.role !==
24   return NextResponse.redirect(new URL("/", req.url));
25 }
26
27 // 3. Jika sudah login (Admin atau User), rute lain (seperti /product) silakan le
28 return middleware(req, next);
29 }
30 }
```



- menambahkan role editor

- Tugas 2:





```
</button>  
<button  
  onClick={() => signIn("github", { callbackUrl, redirect: false })}  
  className={style.login_form_item_button}  
  disabled={isLoading}  
>  
  {isLoading ? "Loading..." : "sign in with github"}  
</button>  
</form>
```

Halaman Login

Email

Password

Login

sign in with google

sign in with github

Tidak punya akun? Ke Halaman Register

- Menambahkan Oauth dari github pada halaman login



- Tugas 3:

```
src > utils > db > TS servicefirebase.ts > loginWithExternalProvider
72
73 export async function loginWithExternalProvider(userData: any, callback: any) {
74   try {
75     const q = query(
76       collection(db, "users"),
77       where("email", "=", userData.email),
78     );
79
80     const querySnapshot = await getDocs(q);
81     const data: any = querySnapshot.docs.map((doc) => ({
82       id: doc.id,
83       ...doc.data(),
84     }));
85
86     if (data.length > 0) {
87       userData.role = data[0].role;
88       await updateDoc(doc(db, "users", data[0].id), userData);
89
90       callback({
91         status: true,
92         data: userData,
93       });
94     } else {
95       userData.role = "member";
96       await addDoc(collection(db, "users"), userData);
97
98       callback({
99         status: true,
100         data: userData,
101       });
102     }
103   }
104 }
```

```
await loginWithExternalProvider(data, (result: any) => {
```

- Melakukan composing function agar tidak menghasilkan banyak code

- Tugas 4:

```
<Image
  src={session.user.image}
  alt={session.user.fullname}
  width={42}
  height={42}
  className={styles.navbar__user__image}
  referrerPolicy="no-referrer"
/>
```



KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
<https://jti.polinema.ac.id>

- Melakukan import image bawaan next untuk menghasilkan gambar tanpa delay menggantikan fitur dari img (html)